RUNI – Algorithms – Homework 8

Due date: Sunday 25/01/2025

NOTE: the general homework guidelines published in homework 1 are valid for all homework assignments in this course.

Directions for all problems: The solutions for all the problems in this assignment are based on dynamic programming. If not specified otherwise, in each question you need to define the entries of the table (if is not specified in the question), describe the base cases, formulate a recursive formula and justify its correctness. Explain how the table is filled, and what the required output is. Analyze and justify the time complexity.

Problem 1 (25 pts)

n star-cookies are placed in a row on the table. For all $1 \leq i \leq n$, the weight of cookie i is a positive integer a_i. Cookie Monster would like to collect cookies of maximal total weight. He is allowed to pick-up cookies only according to their order in the row, and only if the weights of the collected cookies form a decreasing sequence in which any two consequent weights have different parity (no two consequent odd numbers or two consequent even numbers).



For example, for the input in the figure (in which a_1=8), Cookie Monster can pick-up the cookies in locations 1,2,5,6 (corresponding to total weight 8+7+6+3 = 24).

In this question, we will help Cookie Monster find, using dynamic programming, what is the maximal weight of cookies he can collect.

For every $1 \leq i \leq n$, denote by $V[i]$ the maximal weight of cookies that can be collected out of the first i cookies in a sequence that includes the i-th cookie, and possibly additional cookies from the first $i-1$ cookies.

1. What is the expression you want to maximize?

$\max_{1 \leq i \leq n} V[i]$

2. Write a dynamic programming formula for calculating $V[i]$. Distinguish between the base and the general case. Explain the formula.

Base: $V[1] = a_1$

General: for all $1 \leq i \leq n$: $V[i] = a_i + \max(0, \max_{\{j:\ j<i,\ a_j>a_i \text{ and has a different parity}\}} V[j])$

Explanation: A sequence that includes the i-th cookie, and possibly additional cookies from the first $i-1$ cookies, has value a_i plus the maximal possible value of a sequence that can come before the i-th value. Such a sequence ends with some a_j that has a different parity (to guarantee no two consequent odd numbers or two consequent even numbers), and a larger value (to guarantee a decreasing sequence).

3. Describe in words the table to be filled by the algorithm, what is the filling order, and what is the time complexity of the algorithm. Explain your answer.

The table is a one-dimension array of size n including the values of $V[i]$ for all $1 \leq i \leq n$.

The first entry is the base case, and for $1 \leq i \leq n$, the i-th entry is filled in $O(i)$.

The total time complexity for filling the table is therefore $O(n^2)$.

4. Explain how to derive the actual set of cookies that should be collected (and not only their total weight).

Add another table Prev, where Prev[i] is the index of the cookie that comes before cookie i in the solution that determines $V[i]$.

Whenever $V[i]$ is calculated, if $V[i] = a_i$ then set Prev[i]=NULL, else set Prov[i] to be the index j for which $V[i]$ is set to be $a_i + V[j]$ (achieving the max value in the formula).

The last cookie in the solution is the one for which $V[i]$ is maximal. The set of cookies is: i, Prev[i], Prev[Prev[i]], etc, till a NULL.

Problem 2 (25 pts)

You are given n items of sizes $s_1, s_2, \ldots, s_n$. You have two knapsacks of capacities V_1, V_2. You want to pack in the knapsacks a maximal number of items.

For example, if the item sizes are 2, 5, 10, 3, 3, 7, 1, 12, 9 and the knapsacks' capacities are 6 and 17, then you can pack in them six items: On the knapsack having capacity 6, put items of sizes 1,2,3. On the knapsack having capacity 17, put items of sizes 3,5,7 (this knapsack is not fully utilized).

Remark: There are additional ways to pack 6 items, but there is no way to pack more than 6.

Suggest an algorithm for calculating the maximal number of items that can be packed (no need to specify which items are packed, only the number).

Directions: For every $0 \le i \le n, 0 \le a \le V_1, 0 \le b \le V_2$, denote by $M[i, a, b]$ the maximal number of items out of the first i items that can be packed in knapsacks of sizes a and b.

The goal is to maximize $m[n, V1, V2]$
Base: for all $k \ge 0, j \ge 0$: $m[0, k, j] = 0$. Also, to make all queries feasible,
for all $0 \le i \le n$, $k < 0$ or $j < 0$: $m[i, k, j] = -1$.
Recursion:
For all $0 \le i \le n$, $0 \le k \le V_1$, $0 \le j \le V_2$:
$m[i, k, j] = max\{m[i-1, k, j], m[i-1, k-si, j] + 1, m[i-1, k, j-si] + 1\}$

Explanation: When $i = 0$ then no item is available for packing and the maximal number of items that can be packed is 0. If some knapsack has negative capacity, then we initialize the corresponding value to -1 to make sure this case will be well defined.

The recursion formula considers packing the i-th item. There are three possibilities:
Not packing it at all - and then $m[i,k,j] = m[i-1,k,j]$.
Pack it on the first knapsack – and then $m[i,k,j] = m[i-1,k-si,j] + 1$ because one more item is packed (the +1) and we do the best we can with the first $i-1$ items and capacities $k - si$ and j.
Pack it on the second knapsack – and then $m[i,k,j] = m[i-1,k,j-si] + 1$ because one more item is packed and we do the best we can with the first $i-1$ items and capacities k and $j - si$.

The table has three dimensions $0 \le i \le n$, $0 \le k \le V_1$, $0 \le j \le V_2$. The case $i = 0$ is the base case. For every i, the values of $m[i,k,j]$ depends only on the values of $m[i-1,*,*]$ (* can be anything), therefore, after filling $m[0,*,*]$ in the initialization, it is possible to fill the 3D table using a simple nested loop (for i=1 to n, for k= 0 to V1, for j= 0 to V2). Each entry is filled in O(1), the total time complexity is $O(n \cdot V_1 \cdot V_2)$.

Problem 3 (25 pts)

We are given a wooden board of length L and cutting points $l_1, \ldots, l_k$ (l_1 is the leftmost point and l_k is the rightmost point). We wish to cut the board to a few smaller boards, with known lengths, according to the cutting points. Dana, the carpenter, demands x dollars for cutting a board of length x (independent of the cutting points). Our goal is to cut the board according to the cutting points and pay Dana **the minimal total payment for her work.**

For example, if $L = 12$ and the cutting points are $(l_1, l_2, l_3) = (3,5,8)$, then if Dana will cut the board from left to right according to the cutting points, the payment will be 12+9+7 = 28. If, however, she will cut the board from the right to the left, the payment will be 12 + 8 + 5 = 25. Finally, if she first cuts the board in the second point, then the payment will be 12 + 5 + 7 = 24.

1. Consider the following greedy algorithm: at each step, cut the board in a way that minimizes the difference between the lengths of the two resulting boards (in other words, cut closer as possible to the middle of the board). Apply the same rule on every piece that includes a cutting point. Prove the optimality of this algorithm or give a counter example.

The greedy algorithm is not optimal.
Consider $l_1 = 5, l_2 = 6, l_3 = 7$ and $L = 12$. The greedy algorithm will choose first l_2, therefore its solution costs 12+6+6=24. However, if we first cut l_3, then l_1 and finally l_2, we get cost 12+7+2=21.

2. Suggest a Dynamic Programming algorithm for finding the minimal cost of cutting the board (define the table, give an explanation, specify the output of the algorithm and analyze the time complexity). Define $M[i,j]$ as the minimal price of preforming the cuts l_i to l_j. Formally, this is the minimal price for cutting a board of length $l_{j+1} - l_{i-1}$, with cutting points $l_i - l_{i-1}, \dots, l_j - l_{i-1}$.

Let $l_0 = 0$ and $l_{k+1} = L$.
For $i > j$, $M[i,j] = 0$.
For $i \leq j$, $M[i,j] = min\{M[i,x-1] + M[x+1,j] + l_{j+1} - l_{i-1}\}\ for\ i \leq x \leq j$.

Explanation: if $i > j$ then we have nothing to cut. Otherwise, since we already cut l_{j+1} and l_{i-1}, we have a board of length $l_{j+1} - l_{i-1}$, therefore the cutting price is now $l_{j+1} - l_{i-1}$, and we choose the cutting point l_x which minimizes the price.

We fill the table row by row, and return $M[1,k]$.

Since an evaluation of M takes $O(k)$, the time complexity is $O(k^3)$.

Problem 4 (25 points)

In this problem we will show use dynamic programming to find a more efficient algorithm than the naïve one for finding a Hamiltonian path. Throughout suppose that the graph is given to us in an adjacency matrix form.

1. Show that the naïve algorithm of searching the entire solution space takes time $\Omega(n!)$.

There are $n!$ ways to order a set of n vertices.

Each such ordering of the vertices $a_1, \dots a_n$ is a Hamiltonian path if for each i:

$$(a_i, a_{i+1}) \in E.$$

2. Our goal will be to find an algorithm for Hamiltonian path running in time $O(2^n \cdot n^3)$. Show that $2^n \cdot n^3 \in o(n!)$.
   Hint: Use Stirling's formula, which states that $\log(n!) \in O(n \log n)$.

We want to show that for all constants C and sufficiently large n:

$$n! > C \cdot 2^n \cdot n^3.$$

Equivalently we want to show that $\log(n!) > \log(C \cdot 2^n \cdot n^3)$.

Note that $\log(C \cdot 2^n \cdot n^3) \leq \log(C \cdot 2^n \cdot 2^n) = \log(C \cdot 2^{2n}) = C' \cdot 2n \log 2 =$

$$C'' \cdot 2n = C'''n$$

for some constants C', C'', C'''.

By Stirling's formula $\log(n!) \in O(n \log n)$.
In other words, $\log(n!) > C' \cdot n \log n$, for some constant C'.
Thus,
$\log(n!) > C' \cdot \log n > C'' \cdot n > \log(C \cdot 2^n \cdot n^3)$.

3. Describe a DP algorithm for finding a Hamiltonian path in G whose time complexity is $O(2^n \cdot n^3)$.

Directions: Given G, Let $S \subseteq V$ be a set of vertices. Define $M[S, s, t]$ to be TRUE if there is a Hamiltonian s-t path in the subgraph induced by S.

The base case is $M[\{a_i\}, a_j, a_k]$ = TRUE only if $a_i = a_j = a_k$ since we are looking for a Hamiltonian path on a subgraph of size 1.
Filling this in takes time $O(n^3)$.

General DP:

$M[S, s, t]$ = TRUE if there exists a $k \in S$ where $M[S - \{t\}, s, k]$ = TRUE and $(k, t) \in E$.

The graph has a Ham. Path iff $M[V, s, t]$ for some s, t in V.

Filling each entry in the table takes time $O(n)$, since we need to iterate over each $k \in S$. Applying the recursive formula for a specific k can be done in time $O(1)$ since the graph is given by an adjacency matrix, and we can look up $M[S, s, t]$ in $O(1)$. Note that in the recursive formula we only reference sets S with a smaller cardinality.

There are $O(2^n \cdot n^2)$ table entries, since each vertex can either be in S or not, and there are $\binom{n}{2} \in O(n^2)$ ways to choose two vertices s and t.

Thus, the total runtime is $O(2^n \cdot n^3)$.